



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический
университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА 1

«Основные конструкции языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков
программирования»

Выполнил: студент гр. ИУ5-34Б

(Подпись) (Олейник В. Д.)
(Ф.И.О.)

Проверил:

(Подпись) (Гапанюк Ю. Е.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026 г.

Задание

Задание: решить биквадратное уравнение в трех парадигмах программирования: функциональной, процедурной и объектно-ориентированной.

Процедурная парадигма

Реализация представляет собой последовательную обработку корней.

```
def solver(a: int, b: int, c: int) -> List[float]:  
  
    d = b ** 2 - 4 * a * c  
  
    if d < 0:  
        return []  
  
    res = []  
  
    t1 = (-b + d ** 0.5) / (2 * a)  
  
    t2 = (-b - d ** 0.5) / (2 * a)  
  
    if t1 > 0:  
        res.append(t1 ** 0.5)  
        res.append(-(t1 ** 0.5))  
    elif t1 == 0:  
        res.append(0.0)  
  
    if d > 0:  
        if t2 > 0:  
            res.append(t2 ** 0.5)  
            res.append(-(t2 ** 0.5))  
        elif t2 == 0:  
            res.append(0.0)  
  
    return sorted(res)
```

Рис. 1 – процедурная парадигма

Объектно-ориентированная парадигма

Внутри создания объекта сделана минимальная валидация правильности ввода данных.

```
class Solver:

    def __init__(self, A: int, B: int, C: int):

        if A == 0:
            raise ValueError('A != 0!')

        self.a = A
        self.b = B
        self.c = C

        self.d: float | None = None

        self.roots: List[float] = []
```

Рис. 2 – метод инициализации объекта

```

4  class Solver:
19  def solve(self) -> None:
20
21      self.d = self.b ** 2 - 4 * self.a * self.c
22
23      if self.d < 0:
24          self.roots = []
25          return
26
27      roots_set = set()
28      sqrt_d = self.d ** 0.5
29
30      t1 = (-self.b + sqrt_d) / (self.a * 2)
31      t2 = (-self.b - sqrt_d) / (self.a * 2)
32
33      cand = (t1,) if self.d == 0 else (t1, t2)
34
35      for t in cand:
36
37          if t > 0:
38
39              roots_set.add(t ** 0.5)
40              roots_set.add(-(t ** 0.5))
41
42          if t == 0:
43
44              roots_set.add(0.0)
45
46      self.roots = sorted(roots_set)
47

```

Рис. 3 – основной метод решения уравнения

```
4  class Solver:
47
48      def __str__(self) -> str:
49
50          if self.roots:
51
52              res = f"Roots: {self.roots}"
53
54          else:
55
56              res = "No roots"
57
58          return res
59
```

Рис. 4 – метод вывода корней уравнения

Функциональная парадигма

```
2 def extract_roots_from_t(t: float) -> tuple[float, ...]:
3     match t:
4         case _ if t < 0:
5             return ()
6         case 0.0:
7             return (0.0,)
8         case _:
9             return (t ** 0.5, -(t ** 0.5))
10
11 def solve_biquadratic(a: float, b: float, c: float) -> tuple[float, tuple[float, ...]]:
12
13     d = b ** 2 - 4 * a * c
14
15     match d:
16         case _ if d < 0:
17             return d, ()
18
19         case 0.0:
20             t = -b / (2 * a)
21             return d, extract_roots_from_t(t)
22
23         case _:
24             sqrt_d = d ** 0.5
25             t1 = (-b + sqrt_d) / (2 * a)
26             t2 = (-b - sqrt_d) / (2 * a)
27
28             all_roots = tuple(sorted(set(extract_roots_from_t(t1) + extract_roots_from_t(t2))))
29             return d, all_roots
30
31 def format_result(d: float, roots: tuple[float, ...]) -> str:
32     match roots:
33         case ():
34             return f"D = {d}. Действительных корней нет."
35         case (root,):
36             return f"D = {d}. Один корень: {root}"
37         case _:
38             return f"D = {d}. Действительные корни: {list(roots)}"
39
```

Рис. 5 – решение в функциональной парадигме